

BINI

Bhasha In Nano Instructions

The Learner's Guide

Learn a complete programming language in one afternoon.
No experience needed. Version 1.1

03

PRINT word

06

hello

FF

HALT

prints hello

Three bytes. That is a whole BINI program.

An open language: free to read, free to implement.

Welcome to BINI

BINI is a very small programming language. A whole program can be as short as **three bytes**, so short that you can write it by hand on paper. The name stands for **Bhasha In Nano Instructions**: *bhasha* means language, and *nano* means very small.

You do not need to know how to program. If you can read a table and count in hex (the next page teaches that), you can learn all of BINI in one afternoon.

The big idea

A BINI program is a short list of numbers called **bytes**. Next to it lives a small program called the **VM** (virtual machine). The VM reads your bytes from left to right, one instruction at a time, and does what each one says.

Think of a TV remote. Each button sends a tiny code, and the TV already knows what the code means. BINI lets you press several buttons in a row, do arithmetic, and even repeat things.

Why is it so small?

Because your program and the VM agree on things in advance. The word *hello* is not stored inside your 3-byte program. It is already in the VM's **string table**, and your program just says "word number 6". It is like two friends who say "the usual" instead of reading out a whole order. It works because both sides share the same table.

HONEST NOTE

BINI is a command language for tiny devices and a good way to learn how computers really work. It is not a general-purpose language: there is no keyboard input, no big memory and no text processing. That is on purpose. It keeps the whole VM small enough to read in ten minutes.

What you will learn

- Read and write bytes in hex, and run your first program
- Print words, your own text, and numbers
- Control devices: GPIO, a fan, a timer
- Do arithmetic, make loops, and make decisions
- Use the assembler so you can write `print hello` instead of hex
- Add your own words to the language, then practise with 8 exercises

Every program and every output in this guide was produced by running the real VM, so what you see is what you will get.

Bytes and hex in five minutes

A **byte** is a number from 0 to 255. In hexadecimal (hex), a byte is always written with two characters. The characters are 0-9 and then A-F, where A=10, B=11, C=12, D=13, E=14, F=15.

Hex	00	01	09	0A	0F	10	1C	64	80	FF
Decimal	0	1	9	10	15	16	28	100	128	255

Converting hex to decimal

Take the first digit times 16, then add the second digit. So **1C** is $1 \times 16 + 12 = 28$.

Numbers bigger than 255

A number above 255 uses **two bytes**, high byte first (this is called big-endian). The two bytes **1C E0** make one number: $28 \times 256 + 224 = 7392$.

Writing bytes

- Put spaces between bytes or not: **03 06 FF** is the same as **0306ff**.
- Upper or lower case both work.
- Anything after a # on a line is a comment and is ignored.
- A quick way to check your maths: in Python, **hex(7392)** gives **'0x1ce0'** and **int('1CE0', 16)** gives 7392.

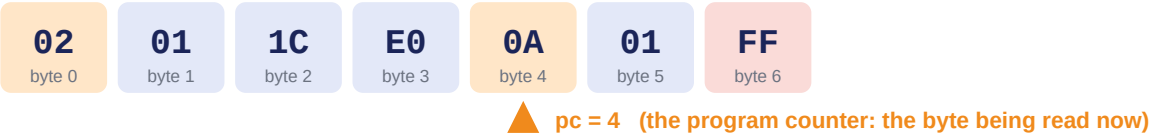
What you need to run BINI

- Python 3.8 or newer (free from python.org).
- The file **bini.py** that comes with this guide. It contains the VM, the assembler, a disassembler and self-tests. The full source is at the end of this guide.
- A terminal. Then try: **python bini.py test** (it should say *all tests passed*).

How the machine works

Here is everything inside the BINI VM, frozen in the middle of running the program **02 01 1C E0 0A 01 FF**:

1 THE PROGRAM (bytes, read left to right)



2 THE REGISTERS (8 boxes, each holds a number from 0 to 65535)



3 THE STRING TABLE (numbers that stand for words)

04	boss	08	off
05	tiny	09	wifi
06	hello	0A	alarm
07	on		
0B and up: your own words			

4 THE OUTPUT (what the program prints)

(nothing printed yet)

The VM repeats four steps until it meets **FF**:

- **Read** the byte at the program counter (pc). That byte is the instruction.
- **Read** the extra bytes (operands) that this instruction needs.
- **Do** the instruction. It might print something, change a register, or move the pc.
- **Move** the pc to the next instruction and go back to step 1.

TIP
Registers all start at 0. There are only 8 of them and no other memory. This is why BINI is so small, and why it cannot do everything Python can.

The instruction set at a glance

BINI has 12 instructions. Learn them in this order; each one has a lesson. The **bytes** column counts the instruction and everything it needs.

Byte	Assembly	Bytes	What it does	Lesson
03	print hello	2	Print a word from the string table	1
01	print "text"	text + 2	Print your own text (01, the letters, then 00)	2
02	load r1 7392	4	Put a number (0-65535) into a register	3
0A	print r1	2	Print the number inside a register	3
04	gpio on	2	Switch the GPIO pin on (01) or off (00)	4
06	fan on	2	Switch the fan on (01) or off (00)	4
05	timer 06:30	3	Set the timer to hours:minutes	4
07	add r2 r0 r1	4	$r2 = r0 + r1$	5
0B	sub r2 r0 r1	4	$r2 = r0 - r1$	5
08	jmp again	2	Continue from another byte	6
09	jz r0 done	3	Jump only if the register is 0	7
FF	halt	1	Stop. Every program ends with this	1

TIP

The **byte** column is the opcode: the first byte of each instruction. The VM decides how many operand bytes to read from that number alone, so there is never any doubt where one instruction ends and the next begins.

Lesson 1: Print a word

Goal: make the computer say hello. The whole program:

03	06	FF
PRINT word	word	HALT

Orange = instruction. **Blue** = extra information the instruction asked for (called operands). **Red** = HALT.

03 is the PRINT-WORD instruction. It needs one more byte: which word to print. **06** is word number 6 in the string table, which is *hello*. **FF** is HALT: it tells the VM to stop. Every program ends with FF.

```
ASSEMBLY
print hello
```

```
$ python bini.py hex "03 06 FF"
hello
```

The default string table

04	05	06	07	08	09	0A
boss	tiny	hello	on	off	wifi	alarm

Several instructions in a row

Put instructions one after another. This prints *hello* then *boss*:

03	06	03	04	FF
PRINT word	word	PRINT word	word	HALT

```
$ python bini.py hex "03 06 03 04 FF"
hello
boss
```

WATCH OUT
FF appears only **once**, at the very end. It is not a full stop after each instruction. It is the HALT instruction.

TRY IT
Change **06** to **04**, **05**, **07**, **08**, **09** or **0A** and run it again. Then try **0B**: error: byte 0: word 0B is not in the string table. (In "Make BINI your own" you will add word 0B yourself.)

Lesson 2: Print your own text

Table words cost one byte, but what if your word is not in the table? Use **text** (instruction **01**). Each letter takes one byte, and a **00** byte marks the end of the text.

01	4E	61	6D	61	73	74	65	00	FF
PRINT text	'N'	'a'	'm'	'a'	's'	't'	'e'	end of text	HALT

Letters are stored as ASCII codes: **4E** is N, **61** is a, **6D** is m, and so on. You never have to look them up if you use the assembler, but it helps to know a few:

Character	space	!	0 to 9	A to Z	a to z
Hex	20	21	30 to 39	41 to 5A	61 to 7A

```
ASSEMBLY

print "Namaste"
```

```
$ python bini.py asm hello.txt
assembled 10 bytes: 01 4E 61 6D 61 73 74
65 00 FF
Namaste
```

TIP
Text costs 1 byte per letter plus 2. *Namaste* takes 10 bytes, while a table word like *hello* takes 3. Use table words when you can.

Lesson 3: Numbers and registers

Numbers live in **registers**: 8 boxes called R0 to R7. **load** puts a number into a register, and **print r1** shows it.

02	01	1C	E0	0A	01	FF
LOAD	reg	high	low	PRINT reg	reg	HALT

02 is LOAD. Then comes the register number (**01** = R1), then the number itself as two bytes: **1C E0** = 7392. **0A** is PRINT-REGISTER, and **01** says which register.

ASSEMBLY

```
load r1 7392
print r1
```

```
$ python bini.py asm num.txt
assembled 7 bytes: 02 01 1C E0 0A 01 FF
7392
```

Words and registers never get mixed up

Printing a *word* (**03**) and printing a *register* (**0A**) are different instructions. So word 05 (*tiny*) and register R5 can live side by side:

02	05	00	2A	0A	05	03	05	FF
LOAD	reg	high	low	PRINT reg	reg	PRINT word	word	HALT

```
$ python bini.py hex "02 05 00 2A 0A 05 03 05 FF"
42
tiny
```

TIP

A register you never loaded holds **0**, so **0A 03 FF** prints 0.

Lesson 4: Devices

Three instructions talk to hardware. Each takes a tiny operand:

Instruction	Operand	Meaning
04 GPIO	00 or 01	00 = off, 01 = on
06 FAN	00 or 01	00 = off, 01 = on
05 TIMER	hour, minute	hour 0-23, minute 0-59 (06 00 = 6:00)

06	01	05	06	00	FF
FAN	0 or 1	TIMER	hour	minute	HALT

ASSEMBLY

```
fan on
timer 06:00
```

```
$ python bini.py asm fan.txt
assembled 6 bytes: 06 01 05 06 00 FF
FAN ON
TIMER 06:00
```

HONEST NOTE

The VM in this guide only *prints* what a device would do. On a real board, whoever ports the VM decides what GPIO, FAN and TIMER actually switch (for example, one pin). BINI v1.1 has no way to name a pin, so one GPIO instruction controls one pin chosen by the device. The last chapter has ideas for growing this.

WATCH OUT

Only 00 and 01 are allowed. **04 02 FF** gives: error: byte 0: expected 00 or 01, got 02

Lesson 5: Math

add r2 r0 r1 reads: **r2 = r0 + r1**. The first register is where the answer goes. **sub** works the same way.

02	00	03	E8	02	01	00	EA	07	02
LOAD	reg	high	low	LOAD	reg	high	low	ADD	dest

00	01	0A	02	FF
a	b	PRINT reg	reg	HALT

ASSEMBLY

```
load r0 1000
load r1 234
add r2 r0 r1
print r2
```

```
$ python bini.py asm sum.txt
assembled 15 bytes: 02 00 03 E8 02 01 00
EA 07 02 00 01 0A 02 FF
1234
```

Numbers wrap around, like an odometer

Registers hold 0 to 65535. Go past the top and you wrap to 0. Go below 0 and you wrap to the top. BINI has no negative numbers.

ASSEMBLY

```
load r0 65535
load r1 1
add r2 r0 r1
print r2
```

```
$ 65535 + 1
0
```

ASSEMBLY

```
load r0 0
load r1 1
sub r2 r0 r1
print r2
```

```
$ 0 - 1
65535
```

TIP

There is no multiply or divide. Multiplying is repeated adding, which you can do with a loop (next two lessons).

Lesson 6: Jumps

Normally the VM reads bytes in order. **JMP** makes it continue from a different byte. Bytes are numbered from 0, and the numbers are shown above each byte here.

byte 0	byte 1	byte 2	byte 3	byte 4
03	06	08	00	FF
PRINT word	word	JMP	byte	HALT

08 00 means "jump to byte 0", so this program prints *hello* forever. The VM refuses to run more than 10,000 steps, so a mistake can never freeze your device:

ASSEMBLY

```
again:
print hello
jmp again
```

```
$ python bini.py hex "03 06 08 00 FF"
error: stopped after 10000 steps
(infinite loop?)
```

TIP

In assembly you write a **label** like `again:` and the assembler counts the bytes for you. You can also write `@0` to mean "byte 0".

Lesson 7: Decisions and loops

JZ means *jump if zero*: **jz r0 end** jumps to end only when R0 is 0. Together with JMP and SUB it gives you a loop. This program counts down from 3:

byte 0	byte 1	byte 2	byte 3	byte 4	byte 5	byte 6	byte 7	byte 8	byte 9
02	00	00	03	02	01	00	01	0A	00
LOAD	reg	high	low	LOAD	reg	high	low	PRINT reg	reg
byte 10	byte 11	byte 12	byte 13	byte 14	byte 15	byte 16	byte 17	byte 18	byte 19
0B	00	00	01	09	00	13	08	08	FF
SUB	dest	a	b	JZ	reg	byte	JMP	byte	HALT

ASSEMBLY

```
load r0 3      # the counter
load r1 1      # the step
loop:
print r0
sub r0 r0 r1   # counter = counter - 1
jz r0 end      # if counter is 0, leave
jmp loop       # otherwise go round again
end:
halt
```

```
$ python bini.py asm countdown.txt
assembled 20 bytes: 02 00 00 03 02 01 00
01 0A 00 0B 00 00 01 09 00 13 08 08 FF
3
2
1
```

Every loop has the same five parts: **(1)** set up a counter, **(2)** do the work, **(3)** subtract 1, **(4)** if the counter is 0 then leave, **(5)** jump back. Here is the VM's own record of the run:

Step	At byte	Instruction	R0	R1	Prints
1	0	load r0 3	3	0	
2	4	load r1 1	3	1	
3	8	print r0	3	1	3
4	10	sub r0 r0 r1	2	1	
5	14	jz r0 @19	2	1	
6	17	jmp @8	2	1	
7	8	print r0	2	1	2
8	10	sub r0 r0 r1	1	1	
9	14	jz r0 @19	1	1	
10	17	jmp @8	1	1	
11	8	print r0	1	1	1
12	10	sub r0 r0 r1	0	1	
13	14	jz r0 @19	0	1	
14	19	halt	0	1	

@19 is byte 19, where the label end sits. @8 is where loop sits.

TRY IT

Change **load r0 3** to **load r0 5**. What does it print? You can watch any program step by step with **python bini.py trace countdown.txt**.

The assembler

Hex is great for learning, but assembly is nicer for real programs. The **assembler** turns readable lines into bytes. Save your lines in a text file and run **python bini.py asm myfile.txt**.

The rules

- One instruction per line. Extra spaces are fine.
- Everything after # is a comment.
- A **label** is a name followed by a colon (loop:). Jumps use the name.
- Capital letters do not matter, except inside quotes: **print "Hi"**.
- Numbers are decimal (500) or hex (0x1F4). Registers are r0 to r7.
- You may leave out the final **halt**. The assembler adds it for you.
- One **print** has three forms: a table word (**print hello**), a register (**print r0**) or quoted text (**print "hi"**).

Reading bytes back

The **disassembler** goes the other way. Give it bytes and it writes assembly. Handy for reading someone else's program:

```
$ python bini.py dis "02 00 00 03 02 01 00 01 0A 00 0B 00 00 01"
0  02 00 00 03    load r0 3
4  02 01 00 01    load r1 1
8  0A 00          print r0
10 0B 00 00 01    sub r0 r0 r1
```

Friendly errors

The assembler tells you which line is wrong and why:

You wrote	The assembler says
<code>gpio onn</code>	line 1: expected on or off, got 'onn'
<code>load r0 5 / print r9</code>	line 2: 'r9' is not a register (use r0-r7)
<code>jmp nowhere</code>	line 1: unknown label 'nowhere'
<code>print bananas</code>	line 1: 'bananas' is not in the string table (for custom text write print "bananas")
<code>timer 25:00</code>	line 1: timer must look like HH:MM (hours 0-23, minutes 0-59)

Make BINI your own

The string table is the part of BINI you are meant to change. Give the assembler and the VM the same table and you have your own vocabulary:

```
from bini import WORDS, assemble, run

my_words = dict(WORDS)
my_words[11] = "door"      # 0B
my_words[12] = "open"     # 0C

program = assemble("print door\nprint open", my_words)
print(program.hex(" "))
print(run(program, my_words))
```

```
03 0b 03 0c ff
['door', 'open']
```

- Numbers **00 to 03** are reserved for future BINI instructions of your own. **04 to 0A** are the default words. **0B to FE** are free for you.
- The sender and the receiver **must use the same table**. A program is only meaningful together with its table. That is the trade: tiny messages, shared vocabulary.

Practice

Try each one on paper first, then check with the VM. Answers are on the next page.

1. Write a **hex** program that prints *wifi* and then *on*.
2. What does **03 0A 03 08 FF** print?
3. Write hex that turns the fan on and sets the timer to 07:30. (Hint: 30 is 1E in hex.)
4. Write hex that prints the number 300. (Hint: 300 = 0x012C.)
5. In assembly, print 500 minus 123.
6. Change the countdown from Lesson 7 so it counts down from 5.
7. Write a loop that prints *hello* three times.
8. Find the bug in each: **(a) 03 06 (b) 04 02 FF (c) 02 08 00 01 FF (d) 08 09 FF**

Answers

- 1. **03 09 03 07 FF** prints wifi, on.
- 2. It prints alarm, off.
- 3. **06 01 05 07 1E FF** prints FAN ON, TIMER 07:30.
- 4. **02 00 01 2C 0A 00 FF** prints 300. (Register R0, high byte 01, low byte 2C.)
- 5. One solution (prints 377):

```
load r0 500
load r1 123
sub r2 r0 r1
print r2
```

- 6. Change only the first line to **load r0 5**. It prints 5, 4, 3, 2, 1.
- 7. Same as the countdown, but print a word instead of the counter:

```
load r0 3
load r1 1
loop:
print hello
sub r0 r0 r1
jz r0 end
jmp loop
end:
halt
```

- 8. The VM finds all four:

Program	The VM says	The fix
03 06	error: program ran off the end (every program must finish with FF)	Add FF at the end
04 02 FF	error: byte 0: expected 00 or 01, got 02	Use 00 or 01
02 08 00 01 FF	error: byte 0: there is no register R8 (use R0-R7)	Use a register from 00 to 07
08 09 FF	error: byte 0: jump to byte 9 is outside the program	Jump to a byte inside the program

Cheat sheet

Byte	Assembly	Bytes	Meaning
01	print "text"	text + 2	print text: 01, letters, 00
02	load rN 500	4	02 R HH LL: register = number
03	print hello	2	03 W: print word W from the table
04	gpio on	2	04 00/01: GPIO off/on
05	timer 06:30	3	05 HH MM: set the timer
06	fan on	2	06 00/01: fan off/on
07	add rD rA rB	4	07 D A B: D = A + B
08	jmp label	2	08 ADDR: continue at byte ADDR
09	jz rN label	3	09 R ADDR: jump if R is 0
0A	print rN	2	0A R: print register R
0B	sub rD rA rB	4	0B D A B: D = A - B
FF	halt	1	stop (end of every program)

String table

04	05	06	07	08	09	0A	0B-FE
boss	tiny	hello	on	off	wifi	alarm	yours

Six rules to remember

- Every program ends with **FF**. FF only ever appears once, as the last instruction.
- Registers are R0-R7, 16-bit (0 to 65535), start at 0, and wrap around.
- Bytes are numbered from 0. Jump addresses are byte numbers.
- Big numbers are two bytes, high byte first.
- A program is at most 255 bytes. Most command programs are 3 to 15 bytes.
- Both sides must share the string table.

Error messages, translated

The VM says	It means
program ran off the end	You forgot FF
unknown instruction XX	That byte is not an opcode. Often an earlier instruction had too few or too many bytes
program ends in the middle of an instruction	An instruction is missing bytes
there is no register R8	Use R0 to R7
expected 00 or 01	GPIO and FAN only take 00 or 01
jump to byte N is outside the program	JMP or JZ points past the last byte
stopped after 10000 steps	Probably an endless loop
word XX is not in the string table	Add it, or use print "text"

Where to go next

A five-day learning path

Day	What to do
1	Read up to Lesson 2. Run 03 06 FF and print your own text.
2	Lessons 3 to 5. Write five programs by hand in hex: a number, a word, text, a device, a sum.
3	Lessons 6 and 7. Trace a loop on paper, then check it with the trace command.
4	Write programs with the assembler. Add three words of your own to the string table.
5	Port the VM to Arduino or C (the same logic in about 60 lines), or write a small compiler that turns English commands into BINI.

Ideas for growing BINI

- **Pin numbers:** let GPIO take a pin, so one program can drive many devices.
- **WAIT:** pause for some milliseconds, so a loop can blink an LED.
- **INPUT:** read a sensor into a register, so programs can react to the world.
- **A message channel:** send 3-byte programs over BLE, LoRa or serial to an ESP32 or Arduino.

Honest notes

- BINI cannot do everything Python can. It has 8 small registers and no other memory. It can loop and decide, which covers many small device jobs.
- "3 bytes" is real for table words because the sender and receiver share the table. When comparing sizes with other languages, compare the whole system.
- The speed of BINI has not been measured. Time it on your own hardware before quoting a number.
- In this VM the devices are simulated: they print what they would do.

Appendix A: What changed from v1.0

Testing v1.0 found some rough edges. Version 1.1 fixes them:

Change	v1.0	v1.1
Printing	03 RR printed a word or a register, so R4-R7 clashed with words 04-07	03 prints words, new 0A prints registers. No clash.
LOAD	02 HH LL, always into R1	02 R HH LL, any register. R0 is usable.
Text	01 ... FF	01 ... 00. FF now only ever means HALT.
TIMER size	Listed as 4 bytes, VM used 3	3 bytes: 05 HH MM.
New instructions	FAN was not in the VM	FAN works. New: ADD 07, SUB 0B, JMP 08, JZ 09.
Registers	R2-R3 for device state, R4-R7 for string numbers (never used)	R0-R7 are all general purpose.
Numbers	16-bit but no wrap-around	Wrap at 65535 and at 0.
Mistakes	Bad input crashed or was ignored	Clear error messages.
Program size	"3 to 15 bytes" and "max 255"	255 is the limit (jump addresses are one byte). 3-15 is a guideline.
Claims	"~80 us" and a size table	Removed until measured.

Appendix B: Specification summary

- **Program:** a sequence of bytes ending with FF (HALT). Written as hex (.bini) or assembly.
- **Registers:** R0-R7, unsigned 16-bit, initially 0, arithmetic wraps modulo 65536.
- **Byte order:** big-endian for the number in LOAD.
- **String table:** byte to word. Default 04-0A. Configurable. Must match between sender and VM.
- **Addresses:** one byte, so programs are at most 255 bytes.
- **Errors:** a VM must reject unknown instructions, cut-off instructions, bad registers, values other than 00/01 for GPIO/FAN, bad timers, jumps outside the program, and running off the end.
- **Safety limit:** the reference VM stops a program after 10,000 steps.
- **Assembler:** labels, comments, decimal and hex numbers. The final halt is added if missing.
- **License:** open, free to implement.

Appendix C: Full source code (bini.py)

This is the complete reference implementation. The file `bini.py` that comes with this guide is the same code, so you do not need to type it.

```

"""BINI v1.1 (Bhasha In Nano Instructions): VM, assembler, disassembler.

Usage:
python bini.py hex "03 06 FF"      run a program written as hex bytes
python bini.py asm prog.txt         assemble a file, show the bytes, run it
python bini.py dis "03 06 FF"      turn bytes back into assembly
python bini.py trace prog.txt       run step by step and watch the registers
python bini.py test                 run the self-tests
"""
import re
import sys

# The default string table. Both the program writer and the VM must share it.
WORDS = {4: "boss", 5: "tiny", 6: "hello", 7: "on", 8: "off", 9: "wifi", 10: "alarm"}

class BiniError(Exception):
    pass

# ----- reading hex
def parse_hex(text):
    """'03 06 FF' or '0306ff' -> bytes. Anything after # on a line is ignored."""
    digits = re.sub(r"#.*", "", text)
    digits = re.sub(r"\s+", "", digits)
    if len(digits) % 2 or not re.fullmatch(r"[0-9a-fA-F]*", digits):
        raise BiniError("hex must be pairs of digits 0-9 A-F")
    return bytes.fromhex(digits)

# ----- VM
def run(program, words=WORDS, max_steps=10_000, trace=None):
    """Run a BINI program. Returns the list of things it printed."""
    R = [0] * 8      # registers R0..R7, each holds 0..65535
    out = []         # everything the program prints
    pc = 0           # program counter: the byte we are reading now
    at = 0           # where the current instruction started
    steps = 0

    def fetch(n):     # read n operand bytes and move pc past them
        nonlocal pc
        if pc + n > len(program):
            raise BiniError(f"byte {at}: program ends in the middle of an instruction")
        chunk = program[pc:pc + n]
        pc += n
        return chunk

    def reg(i):
        if i > 7:
            raise BiniError(f"byte {at}: there is no register R{i} (use R0-R7)")
        return i

    def flag(v):
        if v > 1:
            raise BiniError(f"byte {at}: expected 00 or 01, got {v:02X}")
        return v

    def target(a):
        if a >= len(program):
            raise BiniError(f"byte {at}: jump to byte {a} is outside the program")
        return a

    while True:
        if pc >= len(program):
            raise BiniError("program ran off the end (every program must finish with FF)")
        steps += 1
        if steps > max_steps:
            raise BiniError(f"stopped after {max_steps} steps (infinite loop?)")
        at = pc
        op = program[pc]
        pc += 1

        if op == 0xFF:
            # HALT
            if trace:

```

```

        trace(steps, at, R[:, len(out)])
        break
    elif op == 0x01:
        text = ""
        while True:
            c = fetch(1)[0]
            if c == 0:
                break
            text += chr(c)
        out.append(text)
    elif op == 0x02:
        r, hi, lo = fetch(3)
        R[reg(r)] = (hi << 8) | lo
    elif op == 0x03:
        w = fetch(1)[0]
        if w not in words:
            raise BiniError(f"byte {at}: word {w:02X} is not in the string table")
        out.append(words[w])
    elif op == 0x04:
        out.append("GPIO ON" if flag(fetch(1)[0]) else "GPIO OFF")
    elif op == 0x05:
        hh, mm = fetch(2)
        if hh > 23 or mm > 59:
            raise BiniError(f"byte {at}: timer must be HH 0-23 and MM 0-59")
        out.append(f"TIMER {hh:02d}:{mm:02d}")
    elif op == 0x06:
        out.append("FAN ON" if flag(fetch(1)[0]) else "FAN OFF")
    elif op == 0x07:
        d, a, b = fetch(3)
        R[reg(d)] = (R[reg(a)] + R[reg(b)]) & 0xFFFF
    elif op == 0x08:
        pc = target(fetch(1)[0])
    elif op == 0x09:
        r, a = fetch(2)
        a = target(a)
        if R[reg(r)] == 0:
            pc = a
    elif op == 0x0A:
        out.append(str(R[reg(fetch(1)[0])]))
    elif op == 0x0B:
        d, a, b = fetch(3)
        R[reg(d)] = (R[reg(a)] - R[reg(b)]) & 0xFFFF
    else:
        raise BiniError(f"byte {at}: unknown instruction {op:02X}")

    if trace:
        trace(steps, at, R[:, len(out)])
    return out

# ----- disassembler
_FORMATS = {
    # opcode: (operand bytes, how to show it)
    0x02: (3, lambda r, h, l, w: f"load r{r} {h << 8 | l}"),
    0x03: (1, lambda i, w: f"print {w.get(i, '?')}"),
    0x04: (1, lambda v, w: f"gpio {'on' if v else 'off'}"),
    0x05: (2, lambda h, m, w: f"timer {h:02d}:{m:02d}"),
    0x06: (1, lambda v, w: f"fan {'on' if v else 'off'}"),
    0x07: (3, lambda d, a, b, w: f"add r{d} r{a} r{b}"),
    0x08: (1, lambda a, w: f"jmp @{a}"),
    0x09: (2, lambda r, a, w: f"jz r{r} @{a}"),
    0x0A: (1, lambda r, w: f"print r{r}"),
    0x0B: (3, lambda d, a, b, w: f"sub r{d} r{a} r{b}"),
}

def disassemble(program, words=WORDS):
    """bytes -> list of (address, bytes, assembly text)."""
    rows, pc = [], 0
    while pc < len(program):
        at, op = pc, program[pc]
        pc += 1
        if op == 0xFF:
            text = "halt"
        elif op == 0x01:
            end = program.find(b"\x00", pc)
            if end < 0:
                rows.append((at, program[at:], "?? text never ends"))
                break
            text = 'print "' + program[pc:end].decode("latin-1") + '"'
            pc = end + 1
        elif op in _FORMATS:
            n, fmt = _FORMATS[op]

```

```

        if pc + n > len(program):
            rows.append((at, program[at:], "?? cut off"))
            break
        text = fmt(*program[pc:pc + n], words)
        pc += n
    else:
        text = f"?? unknown {op:02X}"
    rows.append((at, program[at:pc], text))
return rows

# ----- assembler
_ARITY = {"print": 1, "load": 2, "gpio": 1, "fan": 1, "timer": 1,
          "add": 3, "sub": 3, "jmp": 1, "jz": 2, "halt": 0}

def _strip_comment(line):
    in_quote = False
    for i, ch in enumerate(line):
        if ch == '"':
            in_quote = not in_quote
        elif ch == "#" and not in_quote:
            return line[:i]
    return line

def _reg(tok):
    if not re.fullmatch(r"r[0-7]", tok):
        raise BiniError(f"'{tok}' is not a register (use r0-r7)")
    return int(tok[1])

def _num(tok, lo, hi):
    try:
        v = int(tok, 16) if tok.startswith("0x") else int(tok, 10)
    except ValueError:
        raise BiniError(f"'{tok}' is not a number") from None
    if not lo <= v <= hi:
        raise BiniError(f"{tok} is out of range ({lo} to {hi})")
    return v

def _onoff(tok):
    if tok not in ("on", "off"):
        raise BiniError(f"expected on or off, got '{tok}'")
    return 1 if tok == "on" else 0

def _addr(tok, labels, strict):
    if tok.startswith("@"):
        return _num(tok[1:], 0, 255)
    if tok in labels:
        return labels[tok]
    if strict:
        raise BiniError(f"unknown label '{tok}'")
    return 0 # first pass: we only need the size, not the address

def _encode(kind, arg, labels, strict, ids):
    if kind == "lit":
        if not all(0x20 <= ord(c) <= 0x7E for c in arg):
            raise BiniError("text may only use printable ASCII characters")
        return b"\x01" + arg.encode("ascii") + b"\x00"
    op, *a = arg
    if op not in _ARITY:
        raise BiniError(f"unknown instruction '{op}'")
    if len(a) != _ARITY[op]:
        raise BiniError(f"'{op}' needs {_ARITY[op]} value(s), got {len(a)}")
    if op == "halt":
        return b"\xff"
    if op == "print":
        if re.fullmatch(r"r\d+", a[0]):
            return bytes([0x0A, _reg(a[0])])
        if a[0] in ids:
            return bytes([0x03, ids[a[0]]])
        raise BiniError(f"'{a[0]}' is not in the string table "
                        f"(for custom text write print \"{a[0]}\")")
    if op == "load":
        v = _num(a[1], 0, 65535)
        return bytes([0x02, _reg(a[0]), v >> 8, v & 0xFF])
    if op == "gpio":
        return bytes([0x04, _onoff(a[0])])

```

[illegible]

```

assert run(countdown) == ["3", "2", "1"]
assert assemble("print hello") == bytes.fromhex("0306FF")
assert assemble('print "Hi"') == bytes.fromhex("014869" "00FF")
assert [t for _, _, t in disassemble(countdown)][0] == "load r0 3"
assert assemble("\n".join(t for _, _, t in disassemble(countdown))) == countdown
for bad in ("0C FF", "04", "03 06", "08 00", "04 02 FF", "02 08 00 01 FF", "08 09 FF"):
    assert fails(hexrun, bad), bad
for bad in ("gpio onn", "print r9", "jmp nowhere", "print bananas", "fan", "load r0 70000"):
    assert fails(assemble, bad), bad
custom = {**WORDS, 11: "door"}
assert run(assemble("print door", custom), custom) == ["door"]

# ----- CLI
def main(argv):
    if len(argv) < 2 or argv[1] not in ("hex", "asm", "dis", "trace", "test"):
        print(__doc__)
        return 1
    cmd = argv[1]
    if cmd != "test" and len(argv) < 3:
        print(__doc__)
        return 1
    try:
        if cmd == "test":
            selftest()
            print("all tests passed")
            return 0
        if cmd in ("hex", "dis"):
            program = parse_hex(" ".join(argv[2:]))
        else:
            with open(argv[2]) as f:
                program = assemble(f.read())
            print(f"assembled {len(program)} bytes: " + " ".join(f"{b:02X}" for b in program))
        if cmd == "dis":
            for at, chunk, text in disassemble(program):
                print(f"{at:3d} {' '.join(f'{b:02X}' for b in chunk):<14}{text}")
        elif cmd == "trace":
            names = {at: text for at, _, text in disassemble(program)}
            def show(step, at, regs, n_out):
                held = " ".join(f"r{i}={v}" for i, v in enumerate(regs) if v)
                print(f"step {step:3d} byte {at:3d} {names[at]:<18} {held}")
            for line in run(program, trace=show):
                print(" prints:", line)
        else:
            for line in run(program):
                print(line)
    except (BiniError, OSError) as e:
        print("error:", e)
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main(sys.argv))

```